Program 1a.

Aim: Write an ALP to find the sum of first 10 integer numbers

```
        AREA  SUM1TO10 , CODE, READONLY
        EXPORT____main
__main
ENTRY                          ; Mark first instruction to
executeSTART
        MOV R0, #10            ; Starting loop counter value
        MOV R1, #0             ; Result stored in R1, initial value zero
Loop                           ; Calculated 10+9+8+...+1

        ADD  R1, R0            ; R1= R0+R1
SUBS R0, #1                    ; Decrement R0, update flag ("S" suffix)
        BNE   Loop             ; If result in R0 is not zero, jump to
loop
                               ; Result is now in R1
Deadloop
        B  Deadloop            ;forever loop
        END                    ;Mark end of file
```

Result:

Before execution: R1=0x 00 00 00 00

After execution: R1=0x 00 00 00 37

Sum = 10+9+8+7+6+5+4+3+2+1= $(55)_{10}$ = $(37)_{16}$

Program – 1b

Aim: Write an ALP to multiply 2 numbers stored in memory & store the result in memory.

```
        AREA  MULTIPLY , CODE, READONLY
        EXPORT __main
__main
ENTRY       ;Mark first instruction to execute
START
        LDR R0, =VAR1           ;R0 has address of Var1
        LDR R1, [R0]            ;R1= 05

        LDR R0, =VAR2           ;R0 has address of Var2
        LDR R2, [R0]            ;R2=06

        MUL R3, R1, R2          ;R3= (1E)H

        LDR R0, RESULT
        STR R3, [R0]            ;[10000000H]= 1EH

Deadloop
        B  Deadloop             ;forever loop

VAR1 DCD 05
VAR2 DCD 06
RESULT DCD 0X10000000   [1E]
        END                     ;Mark end of file
```

Result:

Before execution: 10000000h: 0X 00 00 00 00

After execution: 10000000h: 0X 1E 00 00 00

Program – 2

Aim: Write an ALP to check if given 32 bit number is even or odd.

```
        AREA  EVENODD , CODE, READONLY
        EXPORT____main
__main
ENTRY                          ;Mark first instruction to executeSTART
        LDR r1,=value          ; r1 loaded with address of value
        LDR r2,[r1]            ; Thru r1 get the number into r2
        RORS r2,r2,#1  ; Rotates R2 to right one position and update  flags
        BCS odd                ; if Carry is set,  jump to label odd
        MOV r9,#00             ;if Carry is not set no. is
        even, Make R9=00
        B back                 ; JUMP to label back
odd     MOV r9,#01             ;no. is odd, Make R9=01
  back  B back                 ;forever loop

value DCD 0x0000000F      ; Input odd No. , Later input an even no.

        END                    ;Mark end of file
```

Result1:

After execution: R9=0x 00 00 00 01 (F=15 is odd no.)

Result2:

Change in Program, Make value = 0x0000000E

After execution: R9=0x 00 00 00 00 (E=14 is even no.)

Program – 3

Aim: Write ALP to set/clear the 3rd bit of a given 32-bit number.

```
    AREA   SETCLEAR, CODE, READONLY
    EXPORT  __main
__main
ENTRY
START
    LDR    R1, =0x00000000    ; Input 32-bit number here for setting bit-3
    MOV    R2, #0x08          ; Mask = 0000 1000 (Mask for bit-3)
     ; -------- SET BIT-3 --------
    ORR    R3, R1, R2         ; R3 = R1 with bit-3 set
     ; -------- CLEAR BIT-3 --------
    LDR    R1, =0xFFFFFFFF    ; Input 32-bit number here for clearing bit-3
    BIC    R4, R1, R2         ; R4 = R1 with bit-3 cleared
STOP
    B      STOP               ; Infinite loop
    END
```

OUTPUT

Input R1 : 0x00000000

Output R3 : 0x00000008 (Bit 3 is Set)

Input R1 : 0xFFFFFFFF

Output R3 : 0xFFFFFFF7 (Bit 3 is Clear)

Note: You can input any number and check any position.

Program – 4

Write ALP to check if a 16-bit number in R0 is negative and perform Sign extension to 32-bit using IT and ORR.

```
        AREA SIGNEXT, CODE, READONLY
        EXPORT __main
__main
ENTRY
START
    LDR   R0, =0x0000F234    ; 16-bit Negative no.  , later take 16-bit positive no 0x00007A2B

    LDR    R1, =0xFFFF0000    ; Mask for sign extension

    LSLS   R2, R0, #16              ; Move bit-15 ? bit-31 (sets N flag)

    IT     MI              ; If negative (N = 1), Minus

    ORRMI  R0, R0, R1        ; Pad 1s from bit-16 to bit-31, for negative number

STOP

    B STOP

    END
```

OUTPUT

R0 = 0000F234 **Result in R0: FFFFF234** (sign bit is extended)

R0 = 00007A2B **Result in R0: 00007A2B** (sign bit is as it is)

Program – 5a

Aim: Write an ALP to store data in RAM

```
        AREA  RAM , CODE, READONLY
        EXPORT____main

__main
ENTRY
START
        LDR R1,VALUE          ;load the address to 10001000H
        MOV R3,#00            ;initialize R3 to 00

        MOV R4,#00            ;initialize R4 to 00

        MOV R5,#6             ; initialize counter r5 by moving 6

        LDR R2,=ARRAY         ;load address of array to R2

LOOP1   LDR R3,[R2]           ;load the value from mem loc pointed by R2 to R3

        STR R3,[R1]           ;store the R3 value to mem locn pointed by R1

        ADD R2,R2,#4          ;increment R2 + 4 times (word operation)

        ADD R1,R1,#4          ;increment R1 + 4 times

        SUBS R5,R5,#1         ; decrement counter R5 by one

        BNE  LOOP1            ;If R5 is not equal to 0 jump to label loop1
back

        B back          ; forever loop
ARRAY DCD 0xAAAAAAAA, 0xBBBBBBBB, 0xCCCCCCCC,
0xDDDDDDDD, 0XEEEEEEEE, 0XFFFFFFFF

VALUE  DCD 0x10001000   ;address of ram assigned to variable 'value'

    END
```

OUTPUT:

Before execution: 10001000h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
After execution: 10001000h: AA AA AA AA BB BB BB BB CC CC CC CC
DD DD DD DD EE EE EE EE FF FF FF FF

Program 5b

Write ALP to find the largest number in an array of five 32-bit numbers

```
        AREA    MAXPROG, CODE, READONLY
        EXPORT __main
__main
ENTRY
START
        MOV     R0, #0              ; R0 = 0 (assume max = 0)
        LDR     R1, =NUM_ARR        ; R1 = pointer to numbers
        MOV     R2, #5              ; Counter = 5 numbers
LOOP
        LDR     R3, [R1], #4        ; Load next number
        CMP     R0, R3              ; Compare R0 - R3
                                    ; If R0 >= R3 ? C = 1
                                    ; If R0 < R3  ? C = 0
        BCS     SKIP            ; If Carry Set ? R0 already bigger
        MOV     R0, R3          ; Else update max

SKIP
        SUBS    R2, R2, #1      ; Decrement counter
        BNE     LOOP            ; Repeat for all 5 numbers

STOP
        B       STOP            ; infinite loop

NUM_ARR DCD     0x00000001, 0x00000005, 0x00000003, 0x00000000, 0xFFFFFFFF
        END
```

OUTPUT

R0 = FFFFFFFF (Max)

Program 6a

Write ALP to demonstrate bit-band operations in Peripheral location using the bit-band alias region.

```
        AREA    BITBAND_DEMO, CODE, READONLY
        EXPORT __main
__main
        LDR    R0, =0x42220180   ; Alias address for GPIOC_ODR bit-0
        MOV    R1, #1
        STR    R1, [R0]          ; Set bit-0
        MOV    R1, #0
        STR    R1, [R0]          ; Clear same bit
STOP    B      STOP
        END
```

Peripheral Bit-Band Mapping Concept
Peripheral region: 0x40000000 – 0x400FFFFF
Alias region: 0x42000000 – 0x43FFFFFF
"Alias Address"=0x42000000+("Byte Offset"×32)+("BitNumber"×4)
Where:

- Byte Offset = Peripheral Address − 0x40000000
- Bit Number = 0...7
- Each byte in the peripheral region → 32 bytes in alias region
- Each bit in the byte → 4 bytes in alias region
- Writing 1 → sets the bit, writing 0 → clears the bit

Example: 0x4001100C, Bit 0 → 0x42220180
Step 1: Peripheral register Address: 0x4001100C Bit: 0
Step 2: ByteOffset= 0x4001100C − 0x40000000 = 0x1100C
Step 3: Alias Address
Alias Address = 0x42000000 + 0x1100C*32 + 0*4
= 0x42000000 + 0x220180
= 0x42220180

Alias Address	Peripheral Address	Bit in Peripheral
0x42220180	**0x4001100C**	**bit-0**

OUTPUT

0x4001100C = 00000001 (it's a 1-bit data, use unsigned int address in memory)
0x4001100C = 00000000

Program 6b. Write ALP to demonstrate pipelining

```
AREA   PIPELINE_DEMO, CODE, READONLY
    EXPORT __main
__main
START
    MOV    R0, #5          ; Load 5 into R0
    MOV    R1, #10         ; Load 10 into R1
    ADD    R2, R0, R1      ; R2 = R0 + R1
    SUB    R3, R2, #2      ; R3 = R2 - 2
    MUL    R4, R3, R0      ; R4 = R3 * R0
STOP
    B      STOP            ; Infinite loop
    END
```

Execution Without Pipelining

Cycle →	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
MOV R0, #5	**F**	**D**	**E**												
MOV R1, #10				**F**	**D**	**E**									
ADD R2, R0, R1							**F**	**D**	**E**						
SUB R3, R2, #2										**F**	**D**	**E**			
MUL R4, R3, R0													**F**	**D**	**E**

F = Fetch **D** = Decode **E** = Execute
Each stage = 1 clock cycle Total stages = 3 Each instruction takes = 3 cycles
Total cycles = 5 × 3 = **15 cycles**

Cycle →	**1**	**2**	**3**	**4**	**5**	**6**	**7**
MOV R0, #5	**F**	**D**	**E**				
MOV R1, #10		**F**	**D**	**E**			
ADD R2, R0, R1			**F**	**D**	**E**		
SUB R3, R2, #2				**F**	**D**	**E**	
MUL R4, R3, R0					**F**	**D**	**E**

Definition
Pipelining is a technique where multiple instruction stages are overlapped to increase CPU throughput.
The **ARM Cortex-M3** uses a **3-stage pipeline**:

1. Fetch
2. Decode
3. Execute

Clock Cycles Required
Total cycles = n + (k − 1)
Where
n = number of instructions = 5
k = number pipeline stages = 3
Total cycles = 5 + (3 − 1)
= **7 cycles**

OUTPUT
R4 = 0x00000041

Result:
Cortex-M3 uses a 3-stage pipeline that improves instruction throughput by overlapping fetch, decode, and execute stages, reducing total execution cycles and increasing speed compared to a non-pipelined processor.